



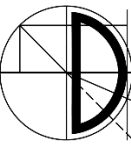
Julius-Maximilians-
**UNIVERSITÄT
WÜRZBURG**

Grundlagen der Programmierung

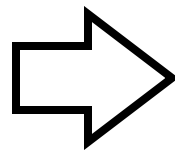
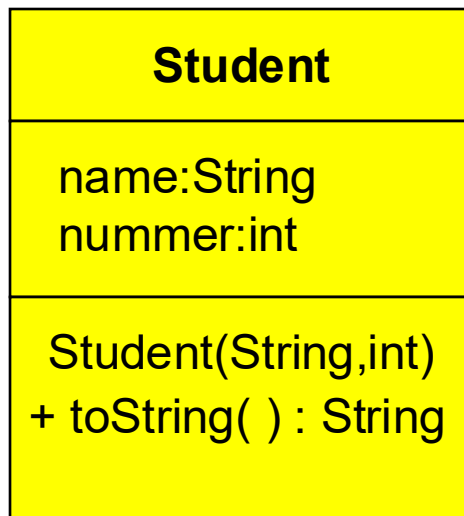
Kapitel 12: Klassen (Teil 2)

Prof. Dr. Samuel Kounev, Daniel Grillmeyer M.Sc.

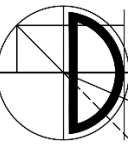
- Alle Materialien (inkl. Vorlesungs- und Übungsfolien, Vorlesungs- und Übungsaufzeichnungen, Übungsblätter, Programmieraufgaben) in diesem Kurs werden nur für Ihre persönliche Nutzung bereitgestellt. Das Teilen oder Weitergeben der Inhalte an Dritte ist generell untersagt!
- Vorlesungsmaterialien von Prof. Dr. Detlef Seese wurden als Basis verwendet
- Unterstützung bei der technischen und inhaltlichen Gestaltung des Vorlesungsmaterials leisteten: Martin Sträßer, Daniel Grillmeyer, Jóakim v. Kistowski, Dietmar Ratz, Joachim Melcher, Roland Küstermann, Jana Weiner, Hagen Buchwald, Matthes Elstermann, Oliver Schöll, Niklas Kühl, Tobias Diederich



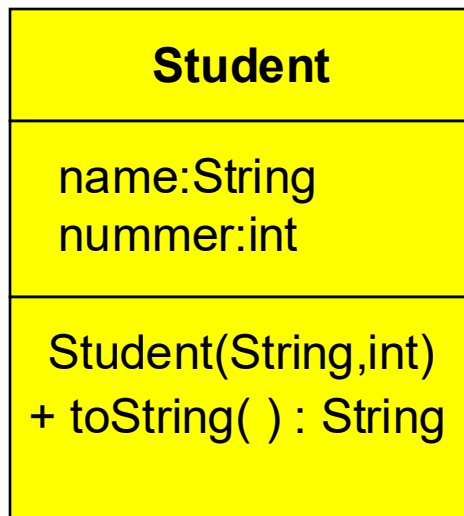
- Letzte Vorlesung: Klasse **Student**
- Code aus abstrakter Klassenbeschreibung (UML-Modell)

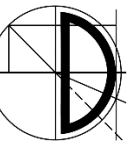


```
public class Student {  
  
    String name;  
    int nummer;  
  
    Student(String n, int nr) {  
        this.name = n;  
        this.nummer = nr;  
    }  
  
    public String toString() {  
        return "Student "  
            +this.name  
            +" hat die Matrikelnummer "  
            +this.nummer  
            +". ";  
    }  
}
```

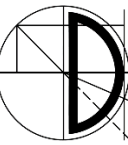


- Heute: Mehr Aspekte aus Java für eine bessere Student-Klasse
- Schlüsselwort **static**

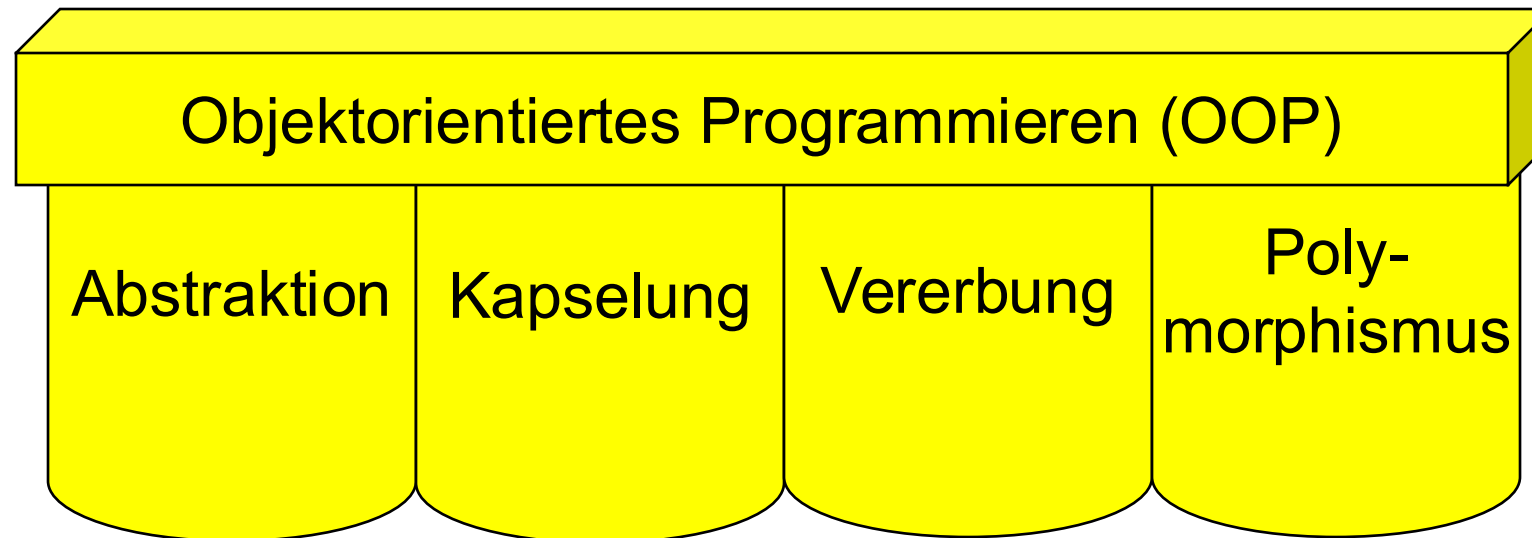




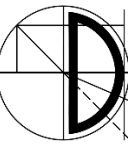
- **Zugriffsrechte unter Klassen**
- **Klassenvariablen und –methoden**



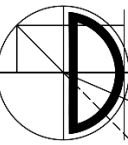
Vier grundlegende Prinzipien des objektorientierten Programmierens:



Als nächstes: **Kapselung**

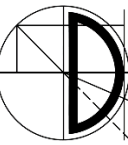


- Kapselung ist die Philosophie, Daten und Methoden fest an ihr zugehöriges Objekt zu binden
- Daten werden
 - dem Benutzer nicht direkt zugänglich gemacht
 - vor Zugriffen von außen geschützt (**information hiding / data hiding**)
- Das Verhalten eines Objekts wird nur durch seine **Schnittstelle** (der nach außen sichtbaren Methoden) bestimmt
- Die Trennung der **Schnittstelle** (der nach außen sichtbaren Methoden) und der **internen Realisierung** bewahrt Programmierer vor Problemen, die sich bei einer internen Umstrukturierung der Klasse ergeben.



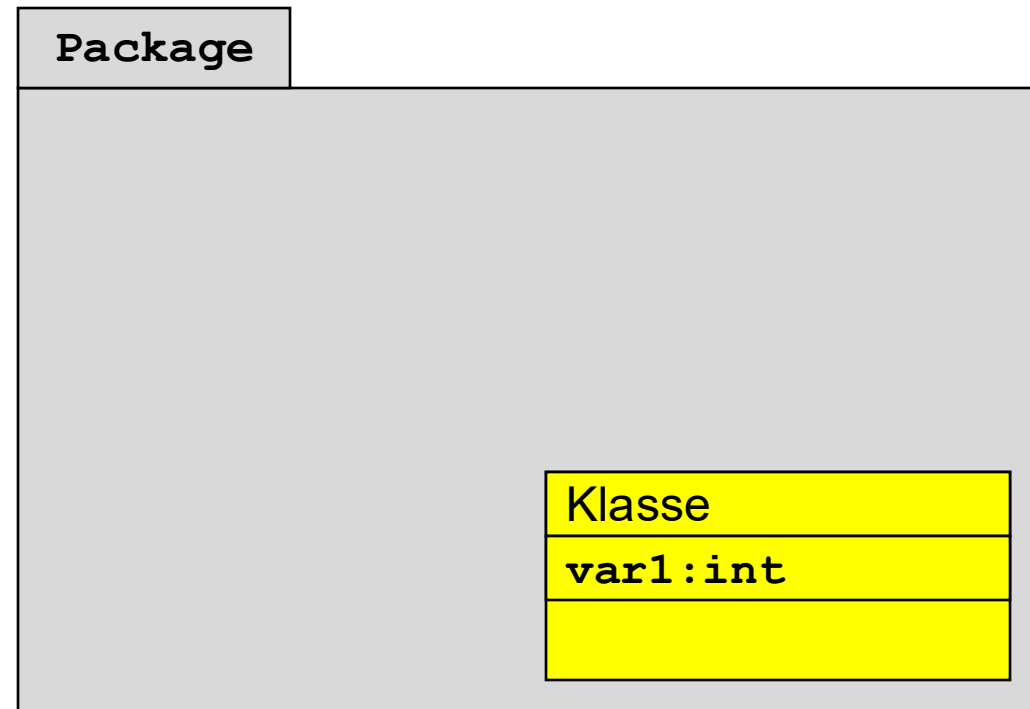
- Bis jetzt, Zugriff auf Instanzvariablen mit:
`<Objektname>.<Variablenname>`
- Beispiel **Student**:
 - Daten (**name** und **nummer**) sowie Methoden (**toString**) sind an ihre zugehörigen Objekte gebunden - eine **Grundforderung** des Prinzips der **Kapselung**
 - Direkter Zugriff auf **name** und **nummer** verstößt gegen die zweite Grundforderung: das **data hiding**
 - **Die Frage ist: Wie können wir unsere Daten besser „verbergen“?**

Student
name:String nummer:int
Student(String,int) + toString() : String



- Java erlaubt **Zugriffs-Modifizierer** (**access modifiers**), davon gibt es vier verschiedene:

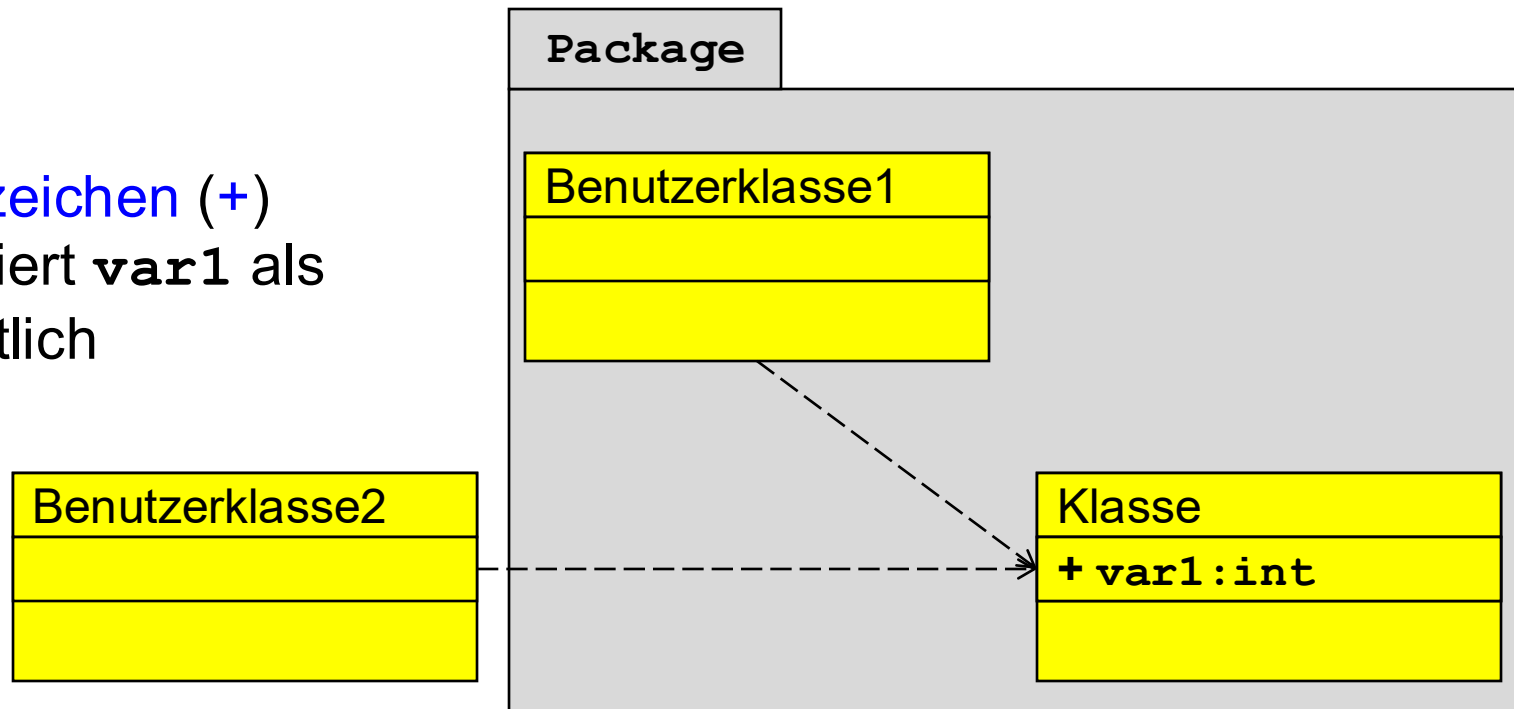
- ① **public**
- ② **private**
- ③ **protected**
- ④ **package**



- **public** (öffentlich): Jede Klasse darf auf eine öffentliche Klasse, Methode oder Variable zugreifen

UML:

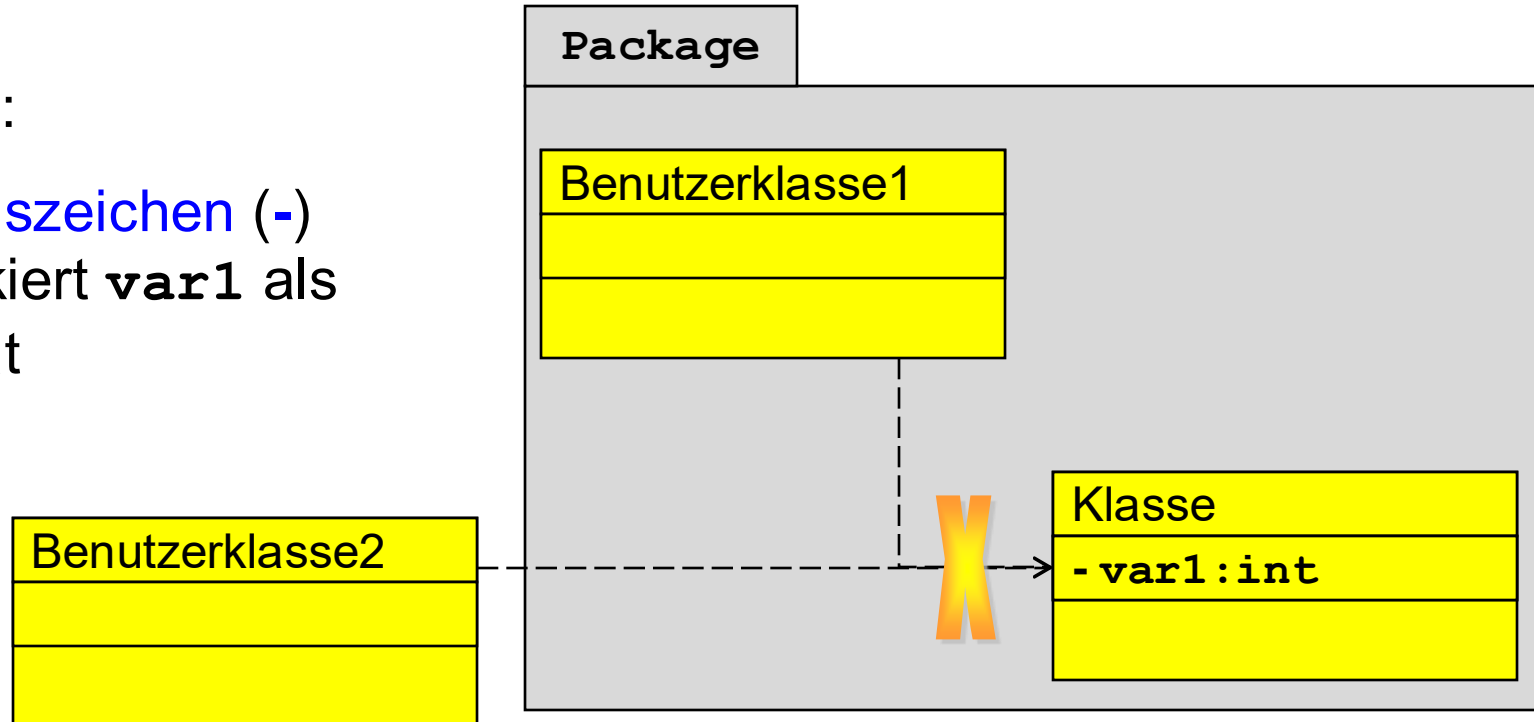
Pluszeichen (+)
markiert **var1** als
öffentlich

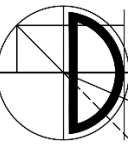


- **private** (privat): Private Methoden und Variablen können **nur innerhalb der Klasse** verwendet werden, in der sie definiert sind (sie sind nur dort sichtbar)

UML:

Minuszeichen (-)
markiert **var1** als
privat



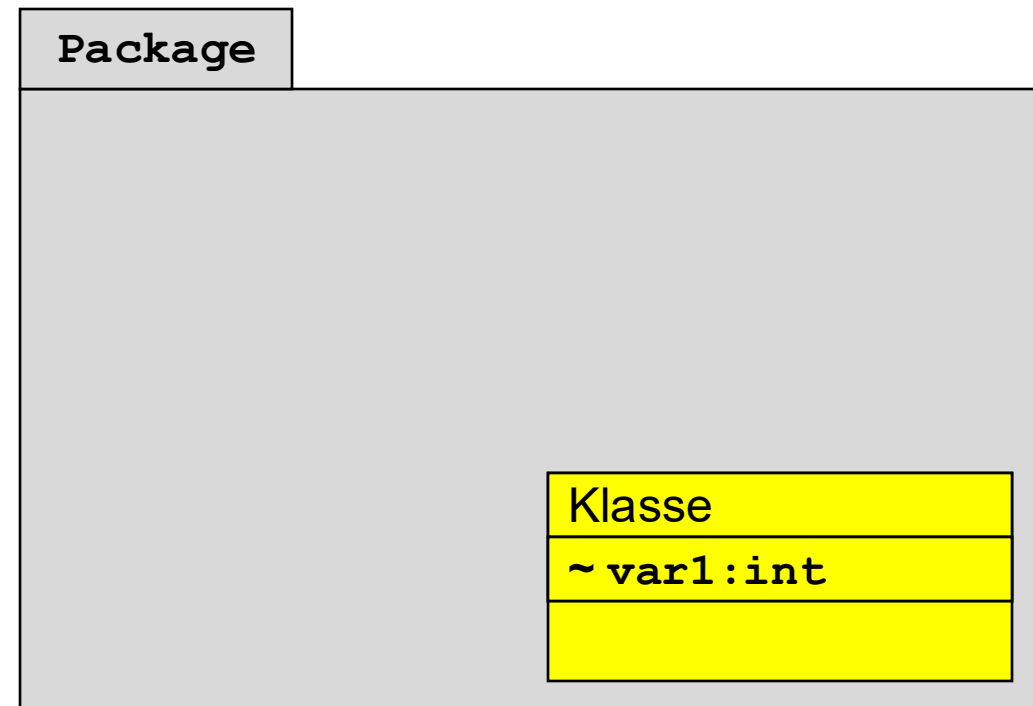


- **package** (Paket): Kann **nur innerhalb des Pakets, in dem sich die Klasse befindet**, verwendet werden
- **Anmerkung:** Dies ist das Standard-Verhalten (kein Modifizierer)

UML:

keine Unterstützung!

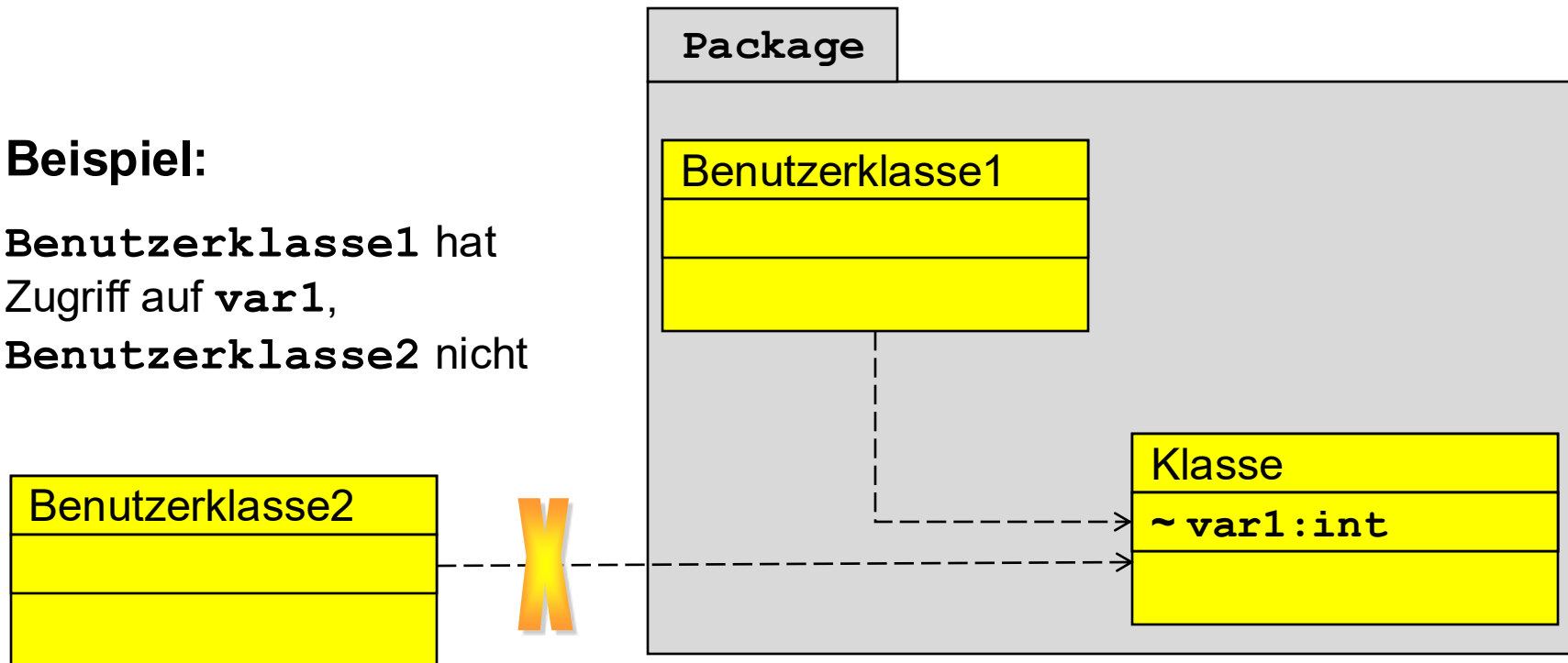
Wir verwenden **Tilde** (~)



- Pakete sind Zusammenfassungen von Klassen
z.B.: `java.lang` oder das `java.util`

Beispiel:

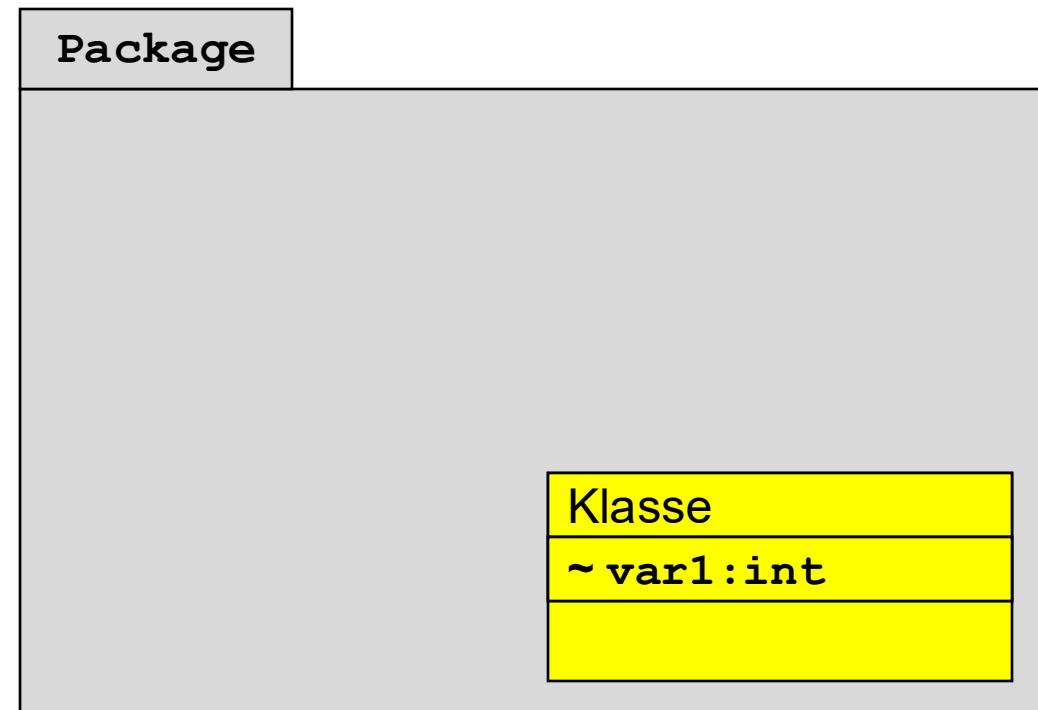
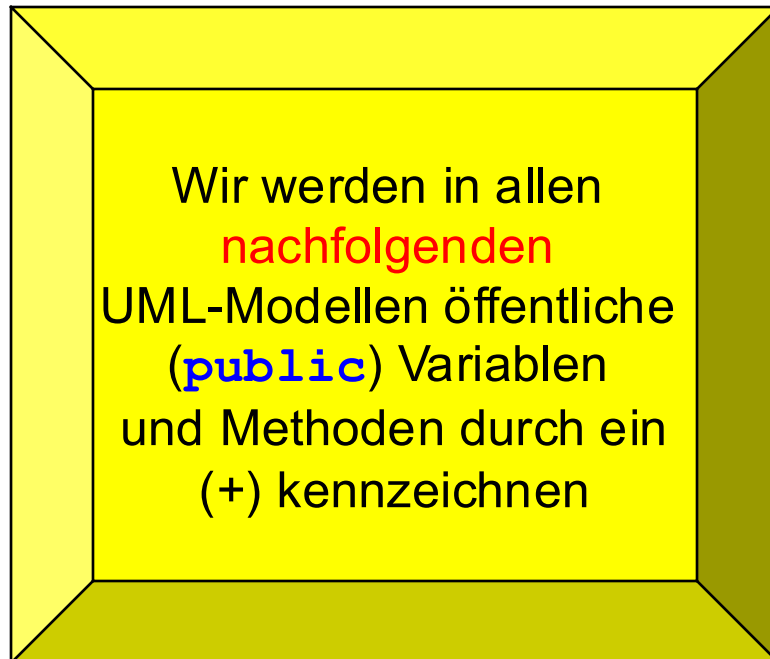
`Benutzerklasse1` hat
Zugriff auf `var1`,
`Benutzerklasse2` nicht



Zugriffsrechte unter Klassen

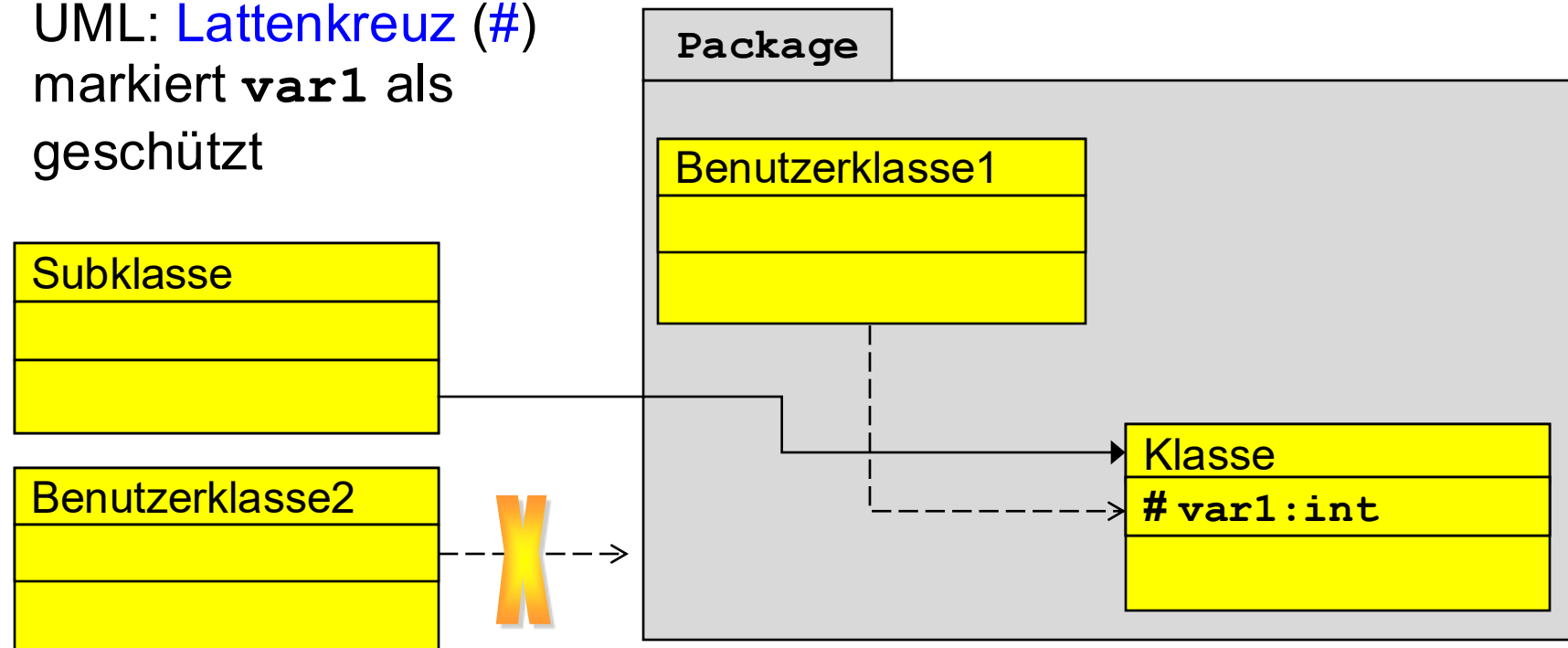
- Standard-Modifizierer von UML und Java sind unterschiedlich:
 - Java: **package**
 - UML: **public**

Wir verwenden Java-Konventionen:

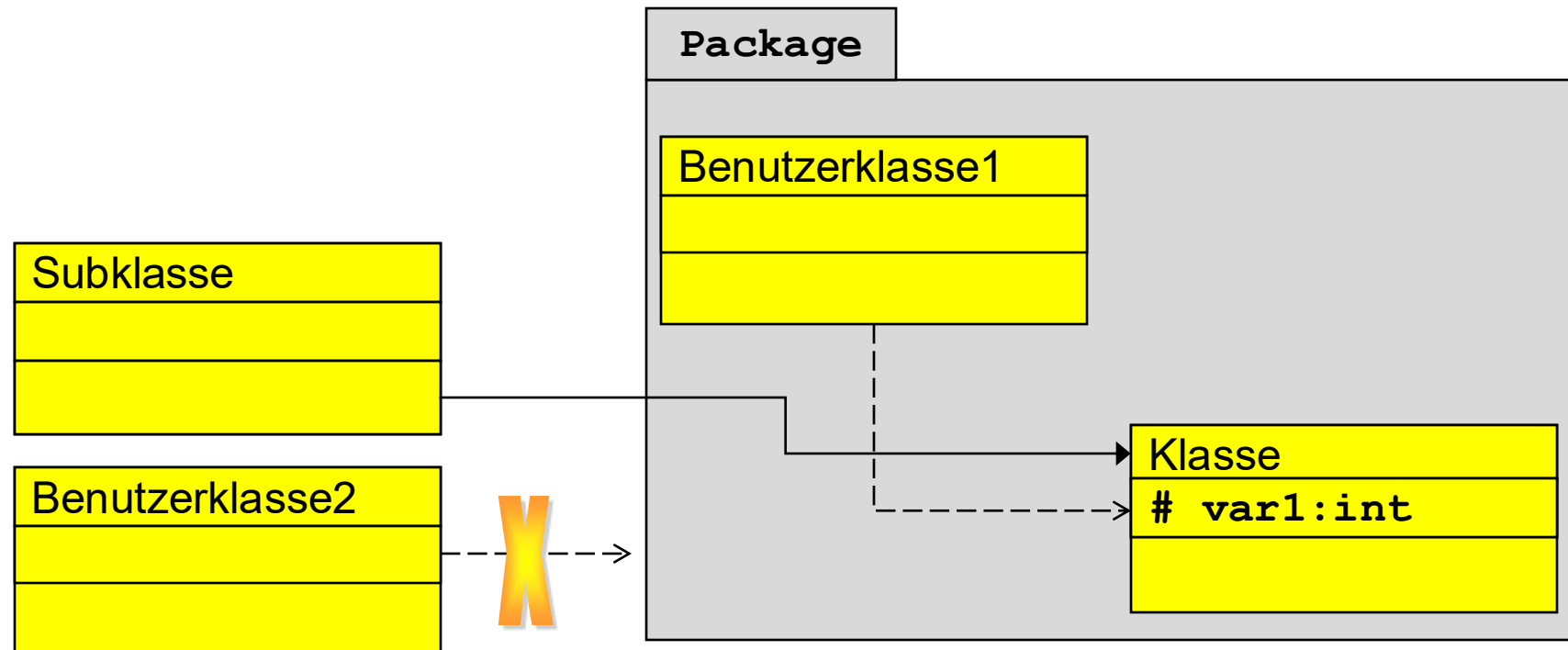


- **protected** (geschützt): Kann **nur innerhalb des Pakets, in dem sich die Klasse befindet** verwendet werden oder von **Subklassen der Klasse** (Subklassen sind erbende Klassen (→ VL 13))

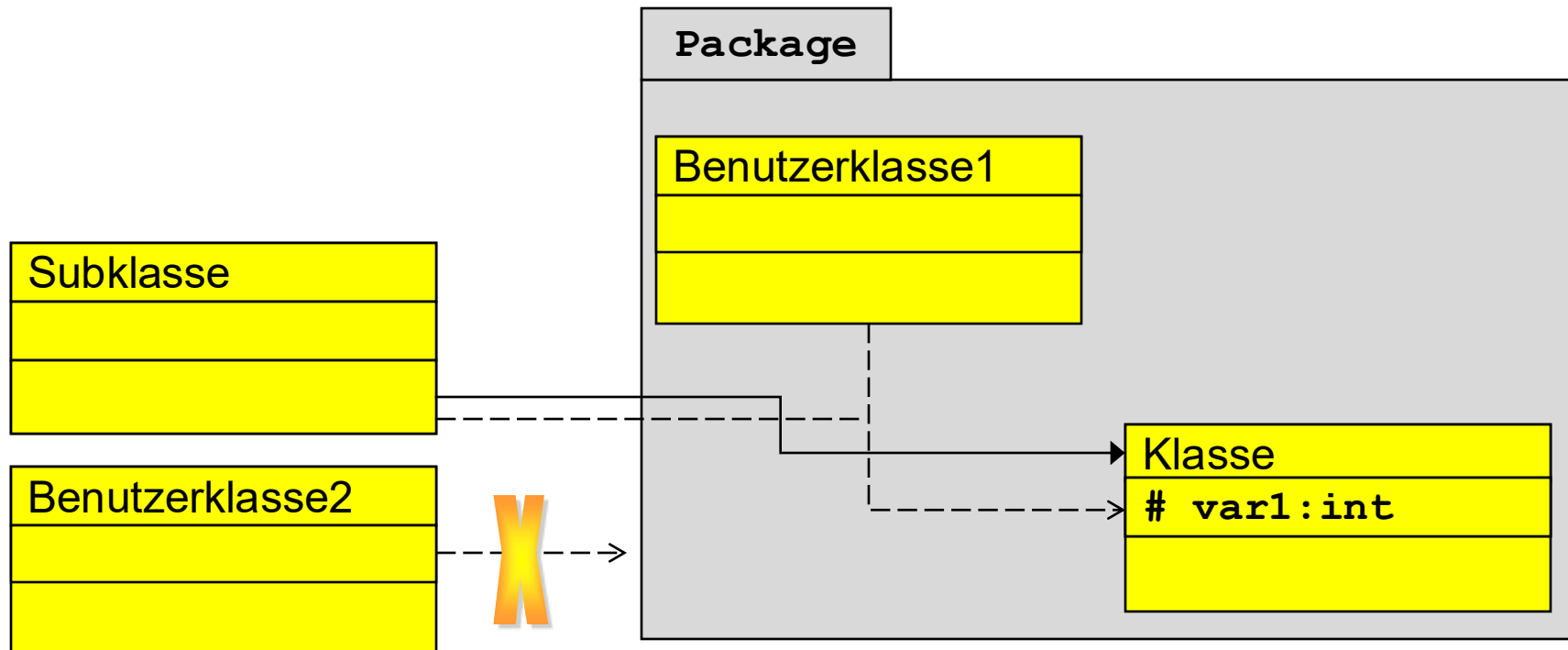
UML: **Lattenkreuz (#)**
markiert **var1** als
geschützt

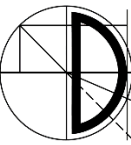


- Der Modus **protected** lässt sich als eine Erweiterung des Standardmodus **package** betrachten. Neben Klassen, die sich in demselben Paket befinden, haben hier auch alle **Subklassen** Zugriff auf ihre Superklasse.

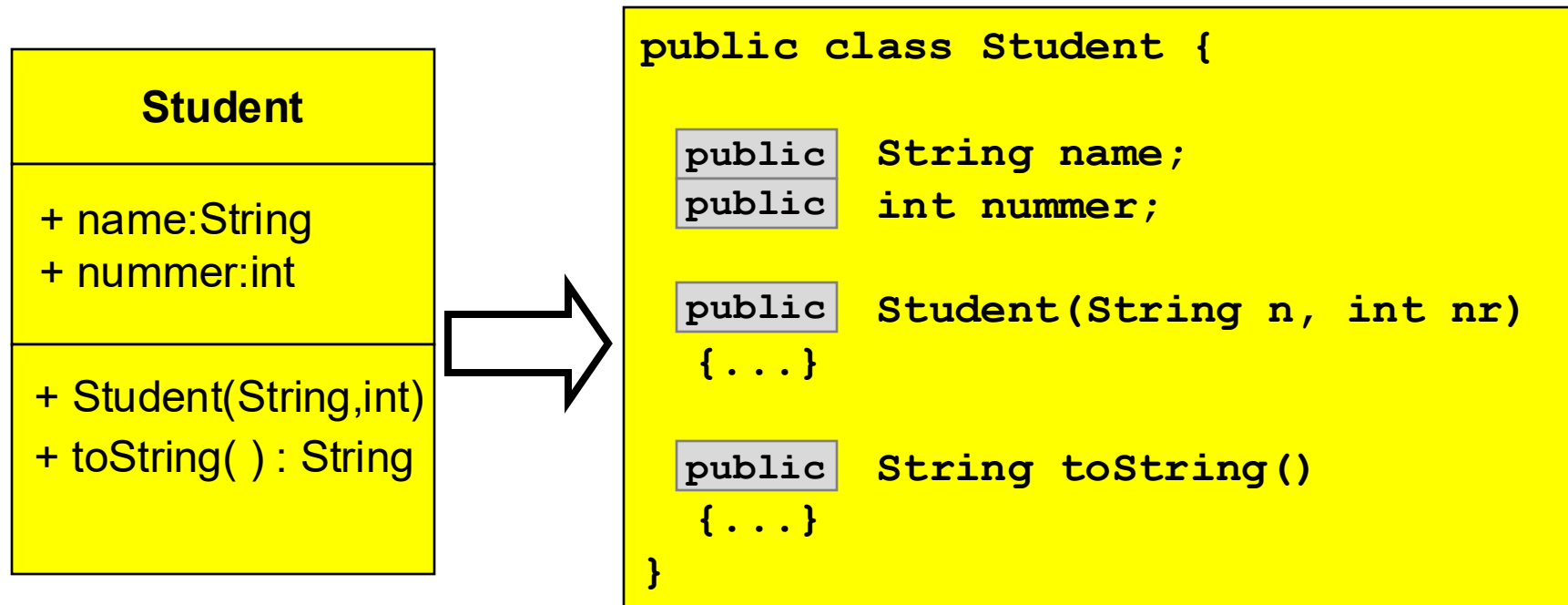


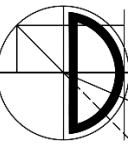
- In unserem Klassenmodell steht **Subklasse** in einer „ist-ein“-Beziehung zur **Klasse**. Sie erhält somit ebenfalls Zugriff auf die Variable **var1**



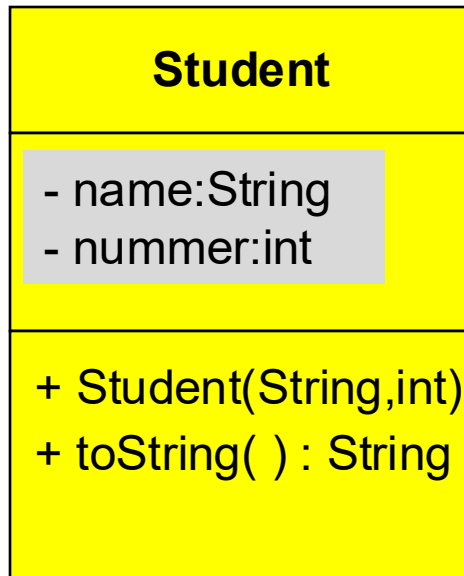


- Wir wollen nun alle Bestandteile von **Student** als **public** deklarieren
- Dazu fügen wir ein: **public** in der Variablendeklaration / Methodensignatur, sowie ein + im UML-Diagramm

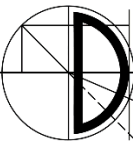




- Gemäß dem Prinzip des **data hiding** wollen wir unsere Instanzvariablen vor einem Zugriff von außen schützen. Wir ändern die Rechte für **name** und **nummer** deshalb in **private** um.



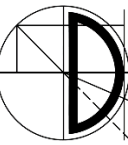
```
public class Student {  
    private String name;  
    private int nummer;  
  
    public Student(String n, int nr)  
    {...}  
  
    public String toString()  
    {...}  
}
```



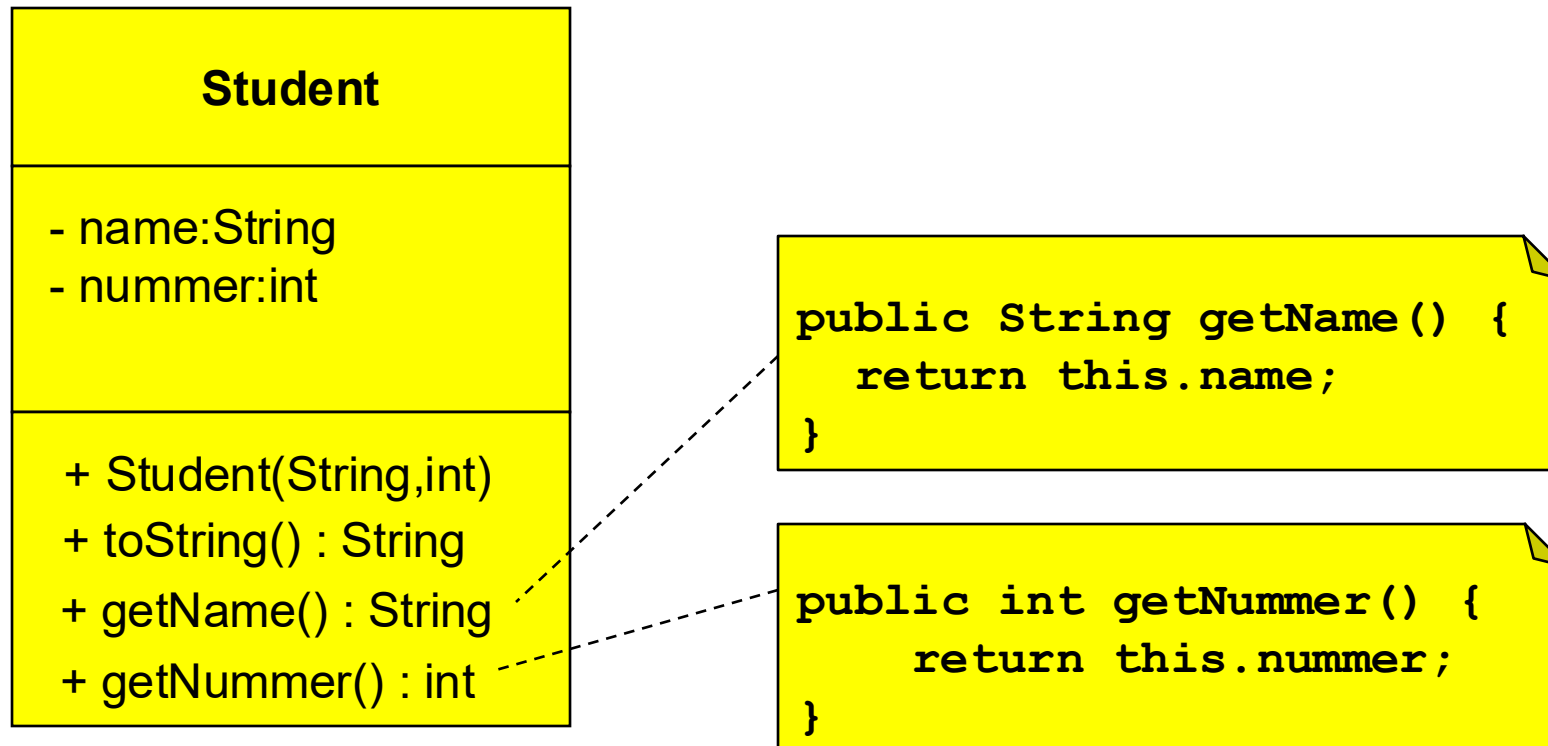
- Unsere Daten sind nun vor unbefugten Zugriffen „geschützt“.
- Problem: Wie kann eine andere Klasse Name und Matrikelnummer erfragen?

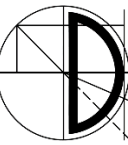
Student
- name:String - nummer:int
+ Student(String,int) + toString() : String

```
public class Student {  
  
    private String name;  
    private int nummer;  
  
    public Student(String n, int nr)  
    {...}  
  
    public String toString()  
    {...}  
}
```

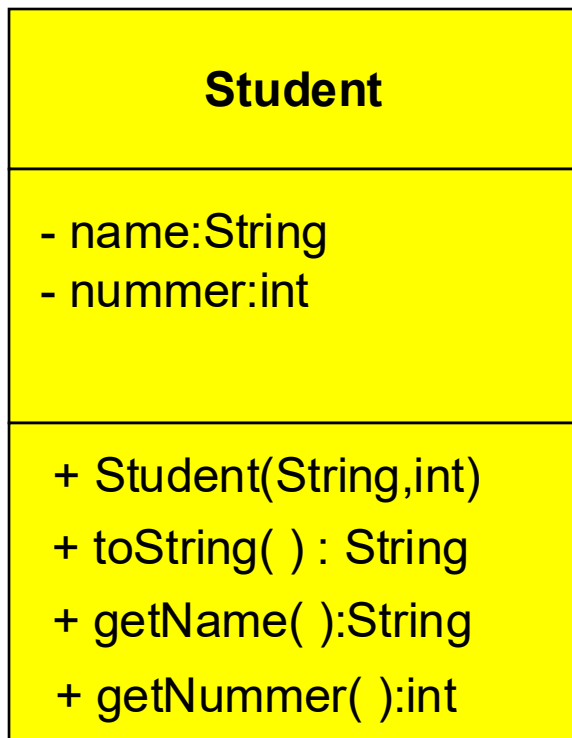


- Zugriff erreichen wir mit Instanzmethoden
 - `getName`
 - `getNummer`



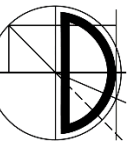


- Aufruf der Instanzmethode `charly.getName()`, um den Namen zu erhalten

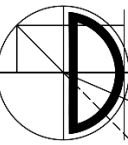


```
String charlysName = charly.getName();  
IO.println(charlysName);
```

```
charly:Student  
name=„Kalle Karlsson“  
nummer=978432
```



- Zugriffsrechte unter Klassen
- **Klassenvariablen und –methoden**

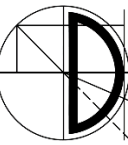


- Wir erweitern die Studentenkartei für den Gebrauch an mehreren Universitäten

Student
- name:String - nummer:int
+ Student(String,int) + toString(): String + getName():String + getNummer():int

Problem:

Name und Matrikelnummer sind nur in **einer** Universität eindeutig. Wir haben keine Garantie dafür, dass es nicht beispielsweise an den Universitäten Würzburg und München zwei **verschiedene** Studenten namens „Kalle Karlsson“ mit Matrikelnummer 978432 gibt. Wie sollen wir diese Studenten voneinander unterscheiden?

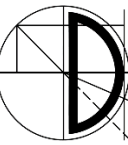


- Zwei Objekte sind gleich, wenn sie **das selbe Objekt** sind (`charly1==charly2`). Dies ist nur genau dann der Fall, wenn ihre Referenzen auf die gleiche Stelle zeigen.

Student
- name:String - nummer:int - id:int
+ Student(String,int) + toString() : String + getName():String + getNummer():int

Jedes **Student**-Objekt soll mit einer eindeutigen Identifikationsnummer (**id**) versehen werden. Die Nummer wird im Konstruktor gesetzt und kann zu Lebzeiten des Objektes nie wieder verändert werden.

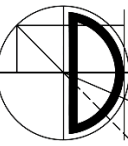
Ziel: Zwei Objekte sollen als gleich gelten, wenn sie dieselbe Nummer besitzen.



- **Problem:** Wie können wir die Instanzvariable `id` mit einem Wert belegen, der für jede Instanz der Klasse `Student` unterschiedlich ist? Woher soll der Konstruktor **wissen**, welche Werte schon vergeben wurden?

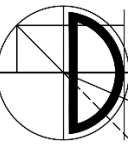
Student
- name:String - nummer:int - id:int
+ Student(String,int) + toString() : String + getName():String + getNummer():int

```
...  
private int id;  
  
public Student(String n,int nr) {  
    this.name=n;  
    this.nummer=nr;  
    this.id = ???  
}  
...
```

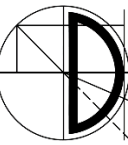


Der Konstruktor unserer Klasse **Student** benötigt Informationen darüber, welche Nummern bereits für andere Objekte vergeben wurden

Allgemeiner: Wie speichern wir eine Information, die nicht zu einem speziellen Objekt, sondern **zu der gesamten Klasse gehört?**



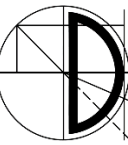
- **Klassenvariablen** und **Klassenmethoden**:
 - Existieren mit der Klasse
 - Werden nicht mit jedem Objekt neu erzeugt
 - Werden auch **statische** Variablen/Methoden genannt
 - Werden mit dem Schlüsselwort **static** deklariert
- Der Zugriff auf statische Teile einer Klasse erfolgt in der Form
<Klassenname>.<Methoden-/Variablenname>
oder in der Form
<Objektname>.<Methoden-/Variablenname>
wobei **<Objektname>** der Name einer Instanz der Klasse ist



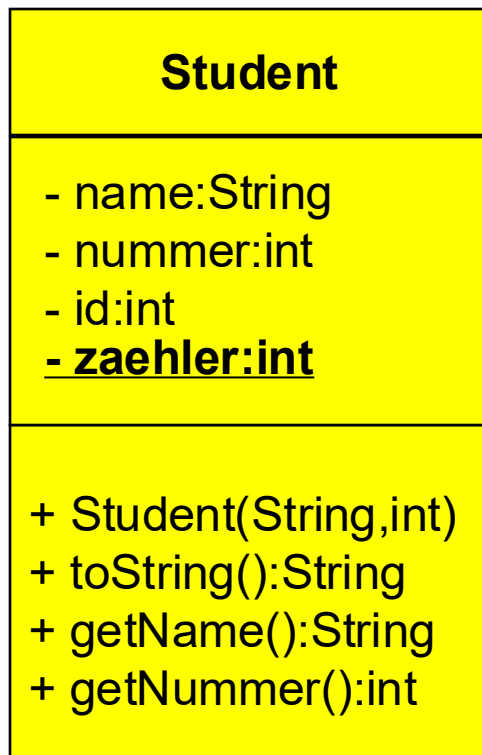
- Konstruktor unserer Klasse **Student**:

Student
- name:String - nummer:int - id:int
+ Student(String,int) + toString():String + getName():String + getNummer():int

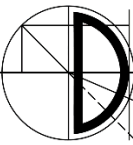
```
...  
private int id;  
  
public Student(String n, int nr) {  
    this.name = n;  
    this.nummer = nr;  
    this.id = ???  
}  
...
```



- Wir speichern die nächste freie ID in einer **Klassenvariable** namens **zaehler**
- UML: statische Elemente werden **unterstrichen**



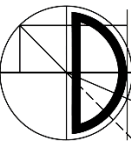
```
...  
private int id;  
private static int zaehler = 1;  
  
public Student(String n, int nr) {  
    this.name = n;  
    this.nummer = nr;  
    this.id = ???  
}  
...
```



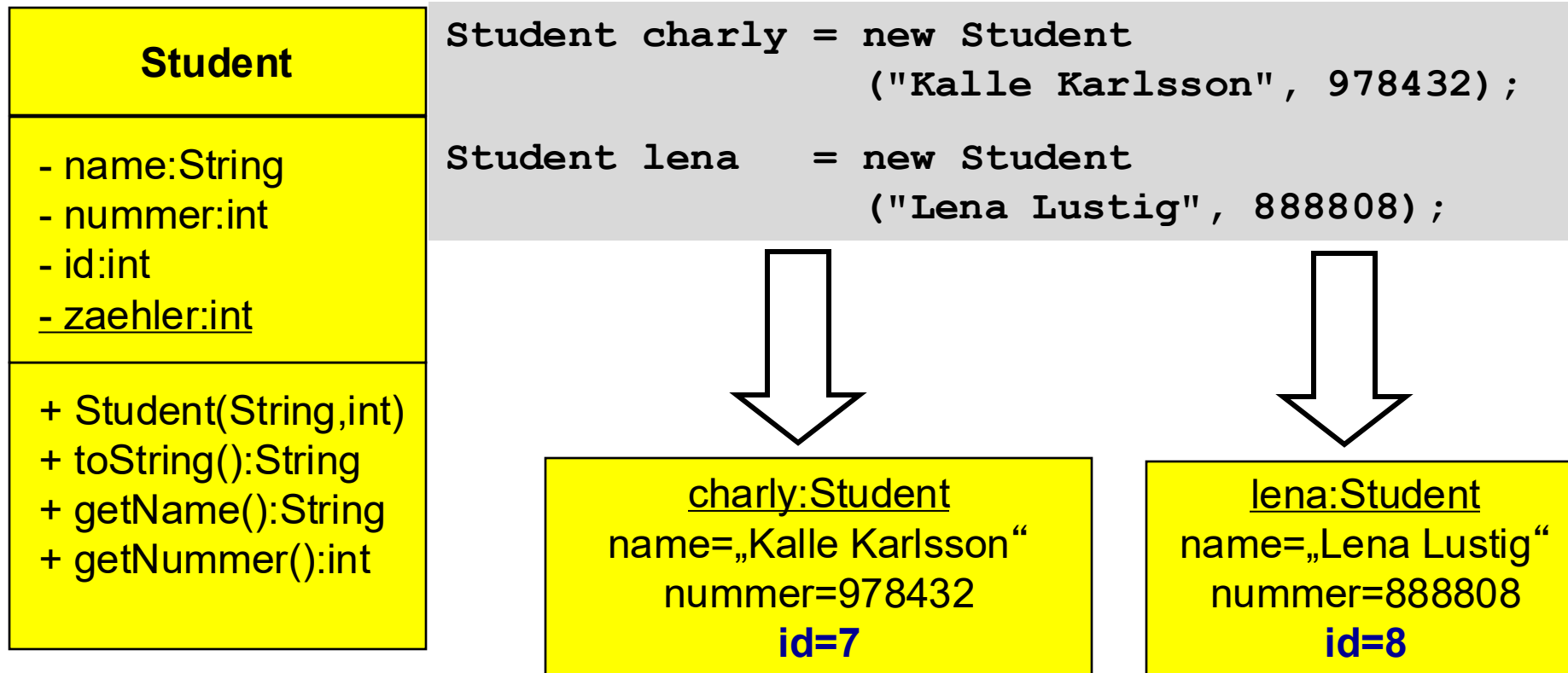
- Statische Klassenvariable **zaehler**
- **id** wird mit dem Wert von **zaehler** initialisiert
- Danach wird der **zaehler** inkrementiert

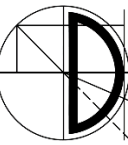
Student
- name:String - nummer:int - id:int - <u>zaehler:int</u>
+ Student(String,int) + toString():String + getName():String + getNummer():int

```
...  
private int id;  
private static int zaehler = 1;  
  
public Student(String n,int nr) {  
    this.name=n;  
    this.nummer=nr;  
    this.id = Student.zaehler;  
    Student.zaehler++;  
}  
...
```

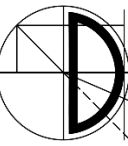


- Die `ids` sind unterschiedlich, obwohl wir an dem Aufruf des Konstruktors keine Veränderungen vorgenommen haben!



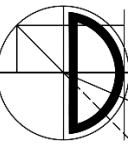


- Die `ids` sind unterschiedlich, obwohl wir an dem Aufruf des Konstruktors keine Veränderungen vorgenommen haben!
- **Information hiding / data hiding!**
 - Interne Darstellung unserer Klasse wurde verändert
 - Alle bisher verfassten Programme lassen sich weiterhin verwenden, weil wir die interne Struktur und Implementierungsdetails **versteckt** und dem Benutzer nur über eine separate **Schnittstelle** zugänglich gemacht haben!

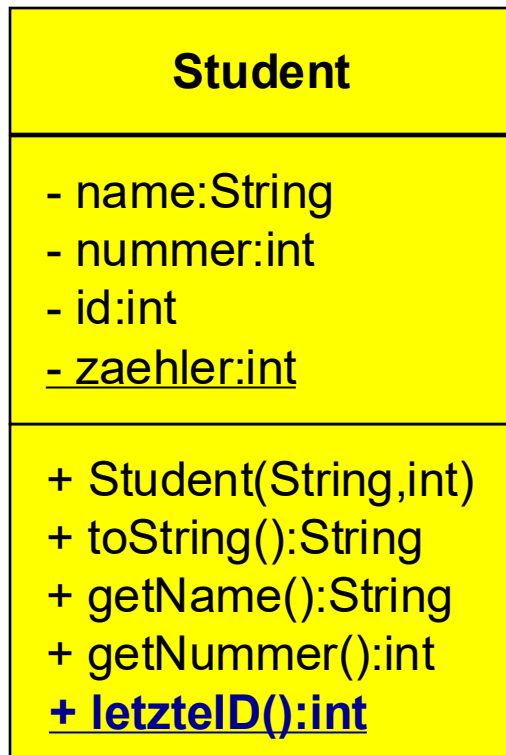


- Wichtige Regel zur Verwendung statischer Methoden:

**Statische Methoden haben keinen
Zugriff auf nicht-statische Elemente!**

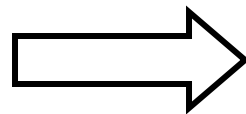


- Statische Methode: **letzteID** soll die letzte vergebene Identifikationsnummer zurückliefern

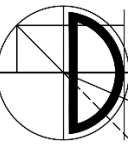


zaehler ist statisch → **letzteID** kann darauf zugreifen

Der Klassenname (**Student**) könnte auch entfallen, weil wir uns in der Klasse **Student** befinden



```
public static int letzteID() {  
    return Student.zaehler - 1;  
}
```



- Die statische Methode **schreibeName** liefert folgenden Compilerfehler:

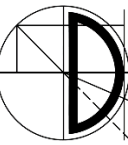
Can't make a static reference to a non-static variable

Student
- name:String - nummer:int - id:int - <u>zaehler:int</u>
+ Student(String,int) + toString():String + getName():String + getNummer():int + <u>letzteID():int</u>

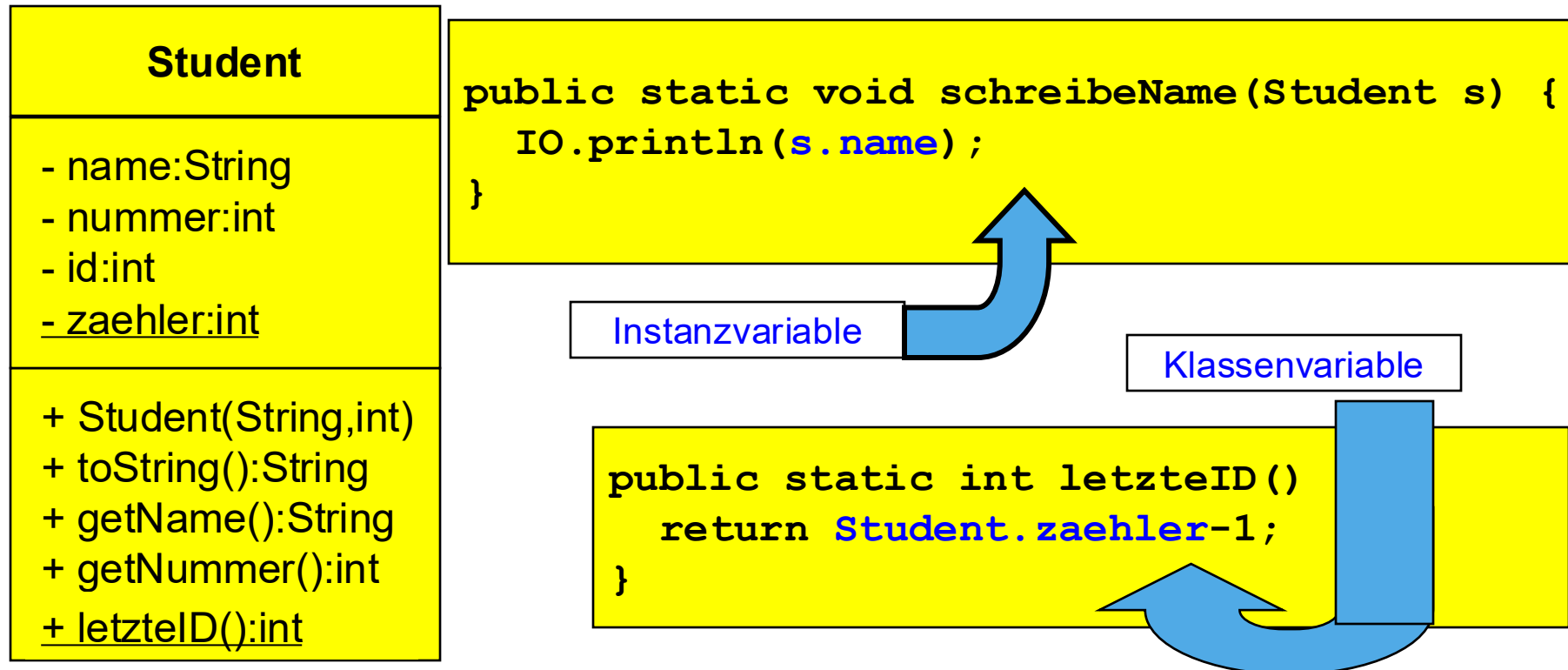
Grund: Zugriff auf Instanzvariable **name**!

Wir befinden uns aber in keiner Instanz (also einem Objekt) der Klasse **Student**, d.h. die Variable existiert nicht!

```
public static void schreibeName() {  
    IO.println(name);  
}
```



- Zugriffe auf **Instanzvariablen** benötigen ein Objekt!
- Zugriffe auf **Klassenvariablen** benötigen **kein** Objekt!



Schon gewusst, dass man in Java auch statische Codeblöcke definieren kann?

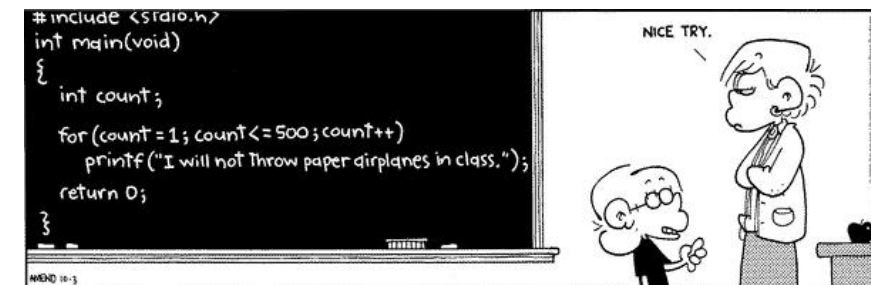
In Java kann man innerhalb einer Klasse (aber außerhalb von Methoden) auch Codeblöcke in folgender Form definieren:

```
static { ... }
```

Diese werden ausgeführt sobald die Klasse zur Laufzeit „geladen“ wird (spätestens wenn diese das erste Mal verwendet/importiert wird). In „normalen“ Klassen treten diese Blöcke nicht auf, spielen aber eine wichtige Rolle in Java-Bibliotheken (z.B. um native Methoden (C-Code) einzubinden)

SOFTWARE TERMINOLOGY

Then	Now
application	app
program	app
operating system	app
script	app
shell	app
batch file	app
compiler	app
daemon	app
service	app
game	app
patch	app
software	app



The image features a background of a classical building facade with columns and arches, likely a part of the Julius-Maximilians-Universität Würzburg. On the left, there is a blue-tinted area with a white logo consisting of a stylized 'J' and 'M' forming a square. To the right of the logo, the text 'Julius-Maximilians-' is in a smaller font, and 'UNIVERSITÄT WÜRZBURG' is in a larger, bold, white font.

Julius-Maximilians-
**UNIVERSITÄT
WÜRZBURG**

Fragen?

Vielen Dank für Ihre Aufmerksamkeit